

NHCC Enterprise Enhancements

Complete Implementation Report

100% COMPLETE

Prepared: July 6, 2026

Project: National Housing & Construction Company — Enterprise Portal

Branch: codex/enterprise-enhancements

Contributors: Andrew Mugoya and Shem Drileba

Overall Implementation Progress

All 8 supervisor-requested phases are now fully implemented.

100%

Before & After Overview

Metric	Before	After
Database Layer	Raw sql.js (WASM, manual SQL)	Prisma ORM with 22 typed models
Audit Trail	None — no history of changes	Auto-logged every create/update/delete with user, IP, and changed fields
Concurrent Edit Protection	None — last save wins	Optimistic locking (version field) + Idempotency-Key headers
Test Data	~30 hardcoded records	2,000+ realistic Ugandan-context records with safe purge

Metric	Before	After
Deployment	Manual — npm start only	Docker Compose (3 services) + GitHub Actions CI/CD
Real-Time Updates	Manual page refresh required	Socket.io live sync — pages update instantly across browsers
File Sharing	No file upload capability	Drag-and-drop uploads with instant broadcast to all users
CEO Reports	None	Auto-generated weekly HTML reports + on-demand via UI button
AI / ML	None	FastAPI service with budget forecasting + risk classification (RandomForest)
Access Control	All users see everything	Role-based sidebar — CEO, Finance, Engineering see different menus
Frontend Pages	10 NHCC pages	14 NHCC pages (+ Audit Logs, Project Detail, Admin Settings, enhanced Reports)
Total Files Changed/Created	—	50+ files across backend, frontend, ML service, and DevOps

Phase 0 — Prisma ORM Migration

100%

Migrated the entire data layer from raw sql.js (SQLite in WASM with handwritten SQL strings) to **Prisma ORM** with a fully typed 22-model schema.

How to see this on the frontend

- Every page that loads data (Projects, Tasks, Messages, etc.) is now powered by Prisma queries instead of raw SQL
- Data loads faster with proper pagination and indexed queries
- The app will feel the same to the user — but the backend is now enterprise-grade

Files Changed

```
[NEW] server/prisma/schema.prisma (22 models, 400+ lines)
[NEW] server/prisma/bootstrap-sqlite.js
[NEW] server/lib/prisma.js (singleton with $extends audit interceptor)
[MOD] server/routes/crud.js (complete rewrite — Prisma queries)
[MOD] server/routes/auth.js (Prisma queries)
[MOD] server/routes/users.js (Prisma aggregations)
[MOD] server/index.js (new route wiring)
```

Phase 1 — Enterprise Audit Trail

100%

Every create, update, and delete operation is now **automatically logged** with full context — who did it, when, from what IP, what browser, and exactly which fields changed.

How to see this on the frontend

Navigate to: Sidebar → **Audit Logs** (visible only to CEO and ICT roles)

Login as: admin@nhcc.go.ug / password123

What you'll see: A searchable table showing every data mutation: Time, User Email, Action (CREATE/UPDATE/DELETE), Entity, Record ID, and Changed Fields.

Files Changed

```
[NEW] server/lib/async-context.js (AsyncLocalStorage for request context)
[NEW] server/middleware/request-context.js
[NEW] server/routes/audit.js (GET endpoint with RBAC)
[NEW] client/src/pages/nhcc/AuditLogs.jsx (by Shem)
[MOD] server/lib/prisma.js ($extends auto-audit)
[MOD] client/src/lib/api.js (auditApi export)
```

Phase 2 — Concurrent Data Protection

100%

Prevents data loss when two people edit the same record simultaneously. Also prevents duplicate form submissions.

How to see this on the frontend

Test scenario: Open the app in two browser tabs. Edit the same project in both. Save in Tab A, then save in Tab B → Tab B will get a "Record was changed by another request. Refresh and try again." error instead of silently overwriting Tab A's changes.

Idempotency: Every POST request from the frontend now sends a unique Idempotency-Key header. If a user accidentally double-clicks Submit, only the first request is processed.

Files Changed

```
[NEW] server/middleware/idempotency.js
[MOD] server/routes/crud.js (version check on PUT)
[MOD] client/src/lib/api.js (Idempotency-Key header on POST)
```

Phase 3 — Realistic Synthetic Data

100%

Generated 2,000+ realistic records using Ugandan contexts (real districts, names, UGX budgets). All synthetic data is flagged with `is_synthetic = true` for safe purging.

How to see this on the frontend

Navigate to: Sidebar → **Admin Settings** (CEO/ICT only)
What you'll see: Two buttons — "Regenerate Data" creates a fresh set of 2,000+ records, and "Purge Synthetic Data" removes only fake records, leaving real data untouched.

Data Generated

Entity	Count	Entity	Count
Projects	110	Invoices	120
Tasks	560	Site Reports	180
Documents	240	Approvals	150
Procurement Records	120	Messages	220
Contracts	120	Calendar Events	160
Users	53	Notifications	180

Files Changed

```
[NEW] server/prisma/seed.js (286 lines, @faker-js/faker)
[NEW] server/routes/seed.js (admin-only POST/DELETE endpoints)
[NEW] client/src/pages/nhcc/AdminSettings.jsx
```

Phase 4 — Production Deployment

100%

Complete containerization and CI/CD pipeline. One command deploys the entire system.

How to use

To deploy: Run `docker compose up --build -d` from the project root.

This spins up 3 containers: Backend (Node.js on port 3001), Frontend (Nginx on port 80), and ML Service (Python on port 8000).

CI/CD: Every push to main automatically runs tests and builds Docker images via GitHub Actions.

Files Created

```
[NEW] server/Dockerfile
[NEW] client/Dockerfile (multi-stage build)
[NEW] client/nginx.conf (reverse proxy for API + WebSocket)
[NEW] docker-compose.yml (3 services)
[NEW] .github/workflows/ci.yml (GitHub Actions)
[NEW] DEPLOYMENT.md (full documentation)
```

Phase 5 — Real-Time Collaboration

100%

When one user creates, edits, or deletes data, all other connected browsers update **instantly** without refreshing.

How to see this on the frontend

Look at: Bottom-right corner of the screen — a green **"Live"** badge confirms the real-time connection is active.

Test it: Open the app in two separate browser windows. Create a new task in Window A → it will appear in Window B within 1 second without refreshing.

Files Changed

```
[NEW] server/lib/socket.js (Socket.io server with rooms)
[NEW] client/src/contexts/SocketContext.jsx (WebSocket client + React Query
invalidation)
[NEW] client/src/components/ConnectionStatus.jsx (Live/Offline badge)
[MOD] client/src/App.jsx (SocketProvider wrapper)
[MOD] server/routes/crud.js (broadcastCrudEvent on every mutation)
```

Phase 6 — Instant File Sharing

100%

Users can now upload documents directly through the browser with drag-and-drop support.

How to see this on the frontend

Navigate to: Sidebar → **Documents**

What you'll see: A dashed upload zone at the top of the page. Drag any file onto it, or click to browse.

Files are saved to the server and a Document record is automatically created. Other connected users see the new document appear instantly via Socket.io.

Files Changed

```
[NEW] server/routes/uploads.js (multer, 50MB limit)
[NEW] server/lib/storage.js (local filesystem adapter)
[MOD] client/src/pages/nhcc/Documents.jsx (drag-and-drop zone)
[MOD] client/src/lib/api.js (documentsApi.upload with FormData)
```

Phase 7 — CEO Automated Reports

100%

Automated weekly HTML report generation with on-demand trigger from the UI. Includes visual charts and KPI cards.

How to see this on the frontend

Navigate to: Sidebar → **Reports** (CEO, Finance, ICT only)

What you'll see:

- Bar chart of Projects by Location (Budget vs Spent in Billions)
- Pie chart of RAG Status Distribution (Green/Amber/Red)
- Pie chart of Task Distribution (by status)
- Financial Summary cards (Total Budget, Total Expenditure, Remaining, Utilization Rate)
- "Generate CEO Report" button — click it to create an HTML report and "Open CEO Report" to view it.

Files Changed

[NEW] server/jobs/ceo-report.js (cron: every Sunday 8AM EAT)

[NEW] server/templates/ceo-report.html

[NEW] server/routes/reports.js (manual trigger endpoint)

[MOD] client/src/pages/nhcc/Reports.jsx (charts + generate button, by Shem)

Phase 8 — Financial Advisor AI (ML Service)

100%

A standalone Python microservice using FastAPI, scikit-learn, and numpy for AI-powered budget forecasting and risk classification.

How to see this

Run: `cd ml-service && pip install -r requirements.txt && uvicorn app.main:app --port 8000`

Open: <http://localhost:8000/docs> for the interactive Swagger API documentation.

Endpoints:

- POST `/api/forecast/budget` — Input: budget, spent, completion%, months elapsed → Output: projected total cost, overrun %, monthly burn rate, recommendation
- POST `/api/risk/classify` — Input: completion%, budget utilization, tasks → Output: predicted RAG status (green/amber/red), confidence score, risk factors, recommendation
- Both support `/batch` for processing multiple projects at once

Files Created

[NEW] `ml-service/app/main.py` (FastAPI entry point)
[NEW] `ml-service/app/routes/forecast.py` (Linear projection model)
[NEW] `ml-service/app/routes/risk.py` (RandomForest classifier)
[NEW] `ml-service/requirements.txt`
[NEW] `ml-service/Dockerfile`

Bonus — Role-Based Access Control (RBAC)

100%

Different user roles see different sidebar menus and are restricted from accessing sensitive pages.

Access Matrix

Page	CEO	Finance	Engineering	Staff
Dashboard, Projects, Tasks, Messages, Documents, Calendar, Announcements, Approvals, Site Reports	✓	✓	✓	✓
Reports & Analytics	✓	✓	Hidden	Hidden
User Management	✓	Hidden	Hidden	Hidden
Audit Logs	✓	Hidden	Hidden	Hidden
Admin Settings	✓	Hidden	Hidden	Hidden

Presentation Login Accounts

Role	Email	Password	Sees
CEO	admin@nhcc.go.ug	password123	Everything
Finance	finance@nhcc.go.ug	password123	All + Reports (no Admin)
Engineering	eng@nhcc.go.ug	password123	Core pages only
Staff	ops@nhcc.go.ug	password123	Core pages only

Final Phase Completion Summary

Phase	Description	Before	After
0	Prisma Migration	0%	100%
1	Audit Trail	0%	100%
2	Concurrency Control	0%	100%
3	Synthetic Data	0%	100%
4	Production Deployment	0%	100%
5	Real-Time Collaboration	0%	100%
6	Instant File Sharing	0%	100%
7	CEO Automated Reports	0%	100%
8	Financial Advisor AI	0%	100%

Project Status: COMPLETE

All 8 phases implemented. 50+ files created or modified across backend, frontend, ML service, and DevOps.

100%